

Introduction to FreeRTOS Multitasking

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

Departamento de Engenharia Informática

Introduction to Multitask FreeRTOS Programming

Good Source to Learn FreeRTOS Programming

Book: [Mastering the FreeRTOS Real Time Kernel - A Hands On Tutorial Guide](#)

What is FreeRTOS?

- A real-time kernel designed specifically for microcontrollers and small microprocessors.
- Developed for over 15 years; now maintained by Amazon Web Services (AWS).
- Distributed under the **MIT Open Source License**, making it free for commercial and academic use.
- **Core Mission:** To provide a small, low-power, and deterministic multitasking environment for embedded systems where reliability is critical.

FreeRTOS Technical Features

Kernel Characteristics

- **Footprint:** Tiny ROM/RAM usage (approx. 6KB to 12KB).
- **Scheduling:** Supports both Pre-emptive and Co-operative modes.
- **Portability:** Written in C; supports over 35 architectures (ARM, ESP32, RISC-V, etc.).

Task Management

- Each task runs in its own context with a private stack.
- The *Scheduler* ensures the highest priority ready task always runs.
- Deterministic resolution via the System Tick.

Why Choose FreeRTOS?

- **Inter-Task Communication:** Robust built-in primitives:
 - ▶ *Queues* for data passing.
 - ▶ *Semaphores/Mutexes* for sync and resource protection.
 - ▶ *Direct-to-task notifications* for high performance.
- **Rich Libraries:** Extensions for TCP/IP, MQTT, HTTP, and Secure Over-the-Air (OTA) updates.
- **Community:** Documentation, active forums, and integration into almost all modern Silicon Vendor SDKs.
- It is a de-facto standard in the industry, ensuring long-term maintainability of embedded codebases.

How Tasks Integrate with FreeRTOS

The Application View

- Tasks are written as independent, infinite C functions.
- They "think" they own the entire CPU.

The Kernel View

- Each task is represented by a **Task Control Block** (TCB).
- Tracks: Priority, Stack Pointer, and Task State.

How Tasks Integrate with FreeRTOS

The Scheduler

The kernel uses a hardware timer (System Tick) to trigger the scheduler:

1. **Save:** Current context to Task A's stack.
2. **Select:** Next Ready Task B via TCB list.
3. **Restore:** Context from Task B's stack.

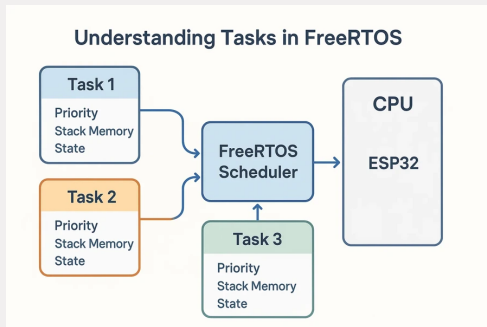


Diagram generated by Gemini.

The Anatomy of a Task

- A task is a C function that **must never return**: it typically contains an infinite loop.
- It is also an independent thread, having its own private stack memory and context.

The Anatomy of a Task

```
void vTaskFunction(void * pvParameters) { // Task entry point (THE Task Function)
    for( ;; ) { // Infinite Loop

        do_something_useful(); // 1. Task Workload Logic

        vTaskDelay(pdMS_TO_TICKS(100)); // 2. Entering Blocked State
    }

    vTaskDelete(NULL); // Safety net: self-deletion
}
```

Instantiating Tasks: xTaskCreate

A task must be instantiated before the scheduler is started. This allocates memory for the task stack and the TCB.

```
BaseType_t xTaskCreate( TaskFunction_t pxTaskCode,  
                        const char * const pcName,  
                        const configSTACK_DEPTH_TYPE usStackDepth,  
                        void * const pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t * const pxCreatedTask );
```

- Returns pdPASS if the task was successfully created.
- Returns errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY if the heap is insufficient.

Task Parameters Explained

- **pxTaskCode:** Pointer to the C function.
- **pcName:** Descriptive string for debugging (not used by the kernel).
- **usStackDepth:** *Critical:* Size of the stack in **words**, not bytes.
- **pvParameters:** Pointer passed into the task as its argument.
- **uxPriority:** 0 is lowest priority.
- **pxCreatedTask:** Handle to reference the task later (can be NULL).

Developer Tip

Always define stack sizes based on your processor architecture. `configMINIMAL_STACK_SIZE` is a good starting point, but complex tasks will require larger buffers.

Launching the Multitasking System

A typical FreeRTOS application follows a standard pattern:

1. define task handles,
 2. instantiate tasks (`xTaskCreate()`), and finally
 3. start the scheduler (`vTaskStartScheduler()`).
- **Task Handles:** Allow us to manipulate tasks (suspend, change priority) after creation.
 - **Priorities:** FreeRTOS uses fixed-priority preemptive scheduling; a higher priority task will preempt a lower priority one whenever the higher priority task enters the ready state.
 - **`vTaskStartScheduler()`:** Never returns; the kernel takes over system timing.

Launching the Multitasking System

```
TaskHandle_t xTaskHandle1, xTaskHandle2;    // Task handles (optional, used for task control)

int main(void) {
    // 1. Instantiate Tasks
    xTaskCreate(vTaskFunction1, "Task1", 1000, NULL, 1, &xTaskHandle1);
    xTaskCreate(vTaskFunction2, "Task2", 1000, NULL, 2, &xTaskHandle2);

    // 2. Start the Scheduler
    vTaskStartScheduler();

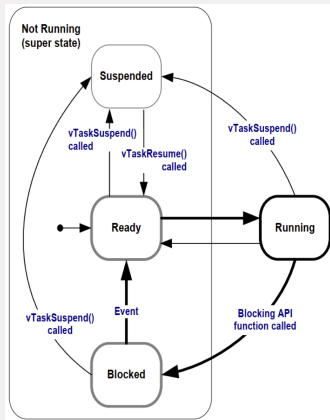
    // Should never reach here
    for(;;);
}
```

The Scheduler

The Scheduler

Pre-emptive Scheduling

- Fixed-Priority Pre-emptive by default.
- Higher priority tasks **pre-empt** lower ones.
- Equal priority tasks share time (Round Robin).



Scheduling Tasks of Equal Priority

When multiple tasks share the same priority and are in the **Ready** state, FreeRTOS manages them based on the `configUSE_TIME_SLICING` configuration.

Scheduling Tasks of Equal Priority

Default Behaviour: Round-Robin with Time Slicing

If `configUSE_PREEMPTION` and `configUSE_TIME_SLICING` are both set to 1:

- The scheduler shares processing time between tasks of equal priority.
- Each task executes for the duration of one **RTOS tick interrupt**.
- When a tick interrupt occurs, the scheduler switches to the next Ready task of the same priority.

Alternative: Without Time Slicing

If `configUSE_TIME_SLICING` is set to 0:

- The scheduler will **not** switch tasks at each tick interrupt.
- A task will run until it enters the *Blocked* or *Suspended* state, or yields voluntarily using `taskYIELD()`.

Scheduling Tasks of Equal Priority

Standard RTOS Practice

In real-time scheduling literature, omitting **time-slicing** for tasks of equal priority is the standard approach.

- **Minimising Overhead:** Disabling time-slicing drastically reduces unnecessary pre-emptions and context switches.
- **Determinism:** It eliminates the CPU execution time wasted on OS house-keeping, ensuring more predictable task execution.
- **FreeRTOS Behaviour:** When time-slicing is disabled, a task will run continuously until it blocks, yields, or is pre-empted by a higher-priority task.

Passing Data to a New Task

- A task can receive data when it is created using the last parameter of `xTaskCreate()`.
- The parameter type is `void *`, allowing any data type to be passed:
 - ▶ Integers
 - ▶ Structures
 - ▶ Arrays
 - ▶ Strings
- Inside the task function, the parameter is recovered using an appropriate cast.

Example Scenario

Goal: Create two tasks that:

- Execute exactly the same task function
- Receive different strings as parameters
- Print their message every 1000 ms

Expected output:

```
Task says: Hello from Task A
Task says: Hello from Task B
Task says: Hello from Task A
Task says: Hello from Task B
...
```

Task Function

```
void vPrintTask(void *pvParameters)
{
    char *message;

    /* Recover parameter */
    message = (char *) pvParameters;

    while(1)
    {
        printf("Task says: %s\n", message);

        vTaskDelay(pdMS_TO_TICKS(1000));
    }

    vTaskDelete(NULL);
}
```

Creating Two Tasks

```
int main(void)
{
    static char msg1[] = "Hello from Task A";
    static char msg2[] = "Hello from Task B";

    xTaskCreate(vPrintTask, "TaskA", 1000, (void *) msg1, 1, NULL);

    xTaskCreate(vPrintTask, "TaskB", 1000, (void *) msg2, 1, NULL);

    vTaskStartScheduler();

    while(1);
}
```

Important Notes

- The same task function can be reused by multiple tasks.
- Each task instance can receive different data.
- The data passed must remain valid after task creation.
- Local variables inside a function may disappear after returning.
- Static or dynamically allocated memory is usually safer.

Task Priorities in FreeRTOS

- The scheduler always executes the highest-priority task that is ready to run.
- Tasks with the same priority share CPU time in turn.
- Priority is assigned during task creation:
`xTaskCreate(..., uxPriority, ...)`
- Priority can also be modified later:
`vTaskPrioritySet()`

Priority Levels

- The number of available priorities is defined by `configMAX_PRIORITIES`
 - ▶ Defined in `FreeRTOSConfig.h`
- Valid priority range: $\{ 0, \dots, \text{configMAX_PRIORITIES} - 1 \}$
- Priority 0 is always the lowest priority.
- Multiple tasks may share the same priority.

Time Measurement and Tick Interrupt

- FreeRTOS measures time using a periodic interrupt **Tick Interrupt**
- The scheduler commonly executes at every tick.
- The time between two consecutive ticks is called **Tick Period**.
- One time slice typically corresponds to one tick period.

Tick Frequency

- Tick frequency is configured in configTICK_RATE_HZ (file FreeRTOSConfig.h).
- Example:

```
#define configTICK_RATE_HZ 100
```

- Therefore a tick occurs every 1/100 seconds, or 10 milliseconds.

Expanding the Not Running State

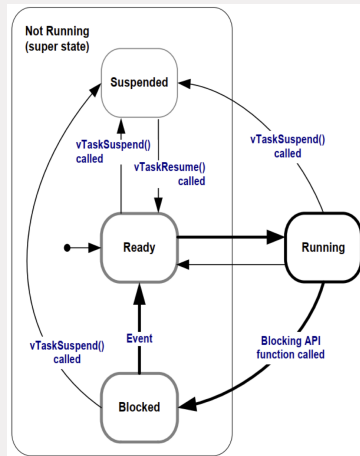
- Tasks that execute continuously are rarely useful.
- If a high-priority task is always running, lower-priority tasks may never execute.
- A better approach is to use **Time-driven** and **Event-driven** tasks.
 - ▶ Time-driven tasks execute periodically, based on a timer or tick count.
 - ▶ Event-driven tasks execute only when an event occurs.

Task States

The Not Running state can be divided into:

- Ready
- Blocked
- Suspended

The Idle task is automatically created when the scheduler starts, ensuring that at least one task is always ready to run.



Blocked State

A task enters the Blocked state while waiting for an event.

Two event types:

1. Temporal events
 - ▶ Delay expiration
 - ▶ Absolute time reached
2. Synchronisation events
 - ▶ Queue data available
 - ▶ Semaphore released
 - ▶ Notification received

Blocking Example

- Task waits for queue data with a timeout:
`xQueueReceive(xQueue, &data, pdMS_TO_TICKS(10));`
- Possible outcomes:
 - ▶ Data arrives (OK)
 - ▶ Timeout expires (error code)
- The task then returns to the Ready state.

Suspended State

- A task enters the Suspended state when it is explicitly suspended by another task or itself.
- Suspended tasks do not consume CPU time and are not scheduled until resumed.
- Enter state: `vTaskSuspend()`
- Exit state: `vTaskResume()` or `xTaskResumeFromISR()`
- Not commonly used in most applications.

Ready State

The task can execute but is waiting for CPU time.

Characteristics:

- Not blocked
- Not suspended
- Waiting for the scheduler to select it

The scheduler always chooses the highest-priority ready task to run.

The vTaskDelay() Function

- vTaskDelay() places the calling task into the Blocked state.
- While blocked:
 - ▶ No CPU time is consumed
 - ▶ Other tasks can execute
- Useful for periodic and event-driven tasks.

Function Prototype

```
[FreeRTOSConfig.h]
#define INCLUDE_vTaskDelay 1

void vTaskDelay(TickType_t xTicksToDelay);
```

Parameter:

- xTicksToDelay
 - ▶ Number of ticks to remain blocked.
 - ▶ The parameter uses **ticks**, not milliseconds.
 - ▶ Use pdMS_TO_TICKS() to convert milliseconds to ticks.

Using Milliseconds

Example: Delay task execution by 100 ms

```
void vTaskFunction(void *pvParameters)
{
    const TickType_t xDelay = pdMS_TO_TICKS(250);

    while(1)
    {
        do_workload();

        vTaskDelay(xDelay);
    }

    vTaskDelete(NULL);
}
```

The vTaskDelayUntil() Function

- Similar to vTaskDelay(), but uses an **absolute** wake-up time instead of a relative delay.
- Ideal for tasks that need to execute at a fixed frequency, ensuring consistent timing even if the task's execution time varies.

Function Prototype

```
void vTaskDelayUntil(TickType_t *pxPreviousWakeTime, TickType_t xTimeIncrement);
```

Parameters:

- pxPreviousWakeTime
 - ▶ Previous wake-up instant
- xTimeIncrement
 - ▶ Task period in ticks

Initialisation

Get the initial clock value before first use:

```
TickType_t xLastWakeTime;  
  
xLastWakeTime = xTaskGetTickCount();
```

Notes:

- Initialization occurs only once
- Value of xLastWakeTime is updated automatically by vTaskDelayUntil()

Periodic Task Example

```
void vTaskFunction(void *pvParameters)
{
    TickType_t xLastWakeTime;

    xLastWakeTime = xTaskGetTickCount();

    while(1)
    {
        do_workload();

        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(TASK_PERIOD_MS));
    }

    vTaskDelete(NULL);
}
```